

Functions as First-Class Objects and Using Them as Arguments

a. What Does “First-Class Objects” Mean?

- In Dart, functions are “first-class objects”, meaning they can be treated like any other variable.
- This allows functions to be assigned to variables, passed as arguments, and returned from other functions.
- This feature enhances flexibility and code reusability.

b. Key Uses of First-Class Functions

- What can you do with functions in Dart?

Feature	Description	Example
Assign to Variables	Store a function inside a variable.	<code>var fn = sayHello;</code>
Pass as Arguments	Send a function as a parameter to another function.	<code>executeFunction(sayHello);</code>
Return from Functions	A function can return another function.	<code>Function getFunction() { return sayHello; }</code>

c. Example: Passing Functions as Arguments

```
void sayHello() {  
  print('Hello!');  
}  
  
void executeFunction(void Function() fn) {  
  fn(); // Call the function passed as an argument  
}  
  
void main() {  
  executeFunction(sayHello); // Pass the sayHello function  
}
```

- Explanation:
 - The function `sayHello()` is passed to `executeFunction()`.
 - Inside `executeFunction()`, the passed function is called using `fn()`.
 - This allows for more dynamic execution of functions.

d. Example: Functions with Parameters

```
void showMessage(String message) {  
  print(message);  
}  
  
void performAction(void Function(String) action, String message) {  
  action(message); // Call the action with the message  
}  
  
void main() {
```

```
performAction(showMessage, 'Button was clicked!'); //
Pass the showMessage function
}
```

- Explanation:
 - showMessage(String message) prints a message.
 - performAction() receives a function (action) and a message.
 - Inside performAction(), the function is executed with the provided message.
- e. Why Is This Useful in Flutter?
 - Passing functions as arguments is particularly useful in Flutter for handling user interactions.
 - For example, buttons in Flutter accept a function as a parameter for the onPressed property.
- f. Advanced Example: Returning Functions

```
Function createMultiplier(int factor) {
    return (int number) => number * factor;
}

void main() {
    var doubleIt = createMultiplier(2); // Returns a function
    that multiplies by 2
    print(doubleIt(5)); // Output: 10
}
```

- Explanation:
 - createMultiplier(int factor) returns a function that multiplies by factor.
 - When createMultiplier(2) is called, it returns a function that doubles numbers.
 - Calling doubleIt(5) results in 10.
- g. Functions in Dart vs. Java
 - Key Differences :

Feature	Dart (Flutter)	Java (Android)
Function Handling	Functions are first-class objects.	Functions are methods inside classes.
Passing Functions	Pass functions as arguments easily.	Typically use interfaces or anonymous classes.
Returning Functions	Can return functions from other functions.	Requires interfaces or lambda expressions.

Ternary Condition Operation

- a. What is a Ternary Condition?
 - A ternary condition is a shortcut for writing an if-else statement in a single line.
 - It checks a condition and quickly decides what to do based on whether it's true or false.
 - It makes the code shorter and easier to read when handling simple conditions.
- b. Basic Syntax